

Hochschule Ravensburg-Weingarten

# Entwicklung eines exemplarischen Crypto-Microservice zur Erklärung von Secure-Coding-Practices

Projektarbeit

im Studiengang Angewandte Informatik

Verfasser: Marcel Eichelberger

Ravensburger Str. 6

Wangen im Allgäu

Matrikelnummer: 35763

Betreuer: Prof. Dr. Sebastian Mauser

RWU Hochschule Ravensburg-Weingarten

13.08.2025

## Hinweis zur geschlechtergerechten Sprache

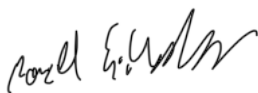
Zur besseren Lesbarkeit werden in dieser Arbeit überwiegend geläufige und sprachlich flüssige Formulierungen verwendet. Personenbezeichnungen gelten, sofern nicht anders kenntlich gemacht, für alle Geschlechter gleichermaßen.

# Eidesstattliche Erklärung

Hiermit versichere ich, dass ich die vorliegende Arbeit selbstständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel verwendet habe und dass die Arbeit in gleicher oder ähnlicher Form keiner anderen Prüfungsbehörde vorgelegt wurde. Alle Ausführungen, die wörtlich oder sinngemäß übernommen wurden, sind als solche gekennzeichnet.

In der Anlage „Dokumentation der Verwendung von KI-Werkzeugen“ habe ich dargelegt, welche KI-Werkzeuge ich genutzt habe, für welchen Zweck ich diese verwendet habe und auf welche Weise die Nutzung stattfand.

Ort, Datum: Wangen im Allgäu, 13.08.2025

Unterschrift: 

# Inhaltsverzeichnis

|                                                               |     |
|---------------------------------------------------------------|-----|
| Hinweis zur geschlechtergerechten Sprache .....               | ii  |
| Eidesstattliche Erklärung .....                               | iii |
| Inhaltsverzeichnis .....                                      | iv  |
| Abbildungsverzeichnis .....                                   | v   |
| 1. Einleitung.....                                            | 1   |
| 1.1. Motivation .....                                         | 1   |
| 1.2. Zielsetzung .....                                        | 1   |
| 1.3. Abgrenzung.....                                          | 2   |
| 2. Grundlagen .....                                           | 3   |
| 2.1. Microservices .....                                      | 3   |
| 2.2. Kryptografie und Datensicherheit .....                   | 4   |
| 2.3. OWASP SCPs .....                                         | 5   |
| 3. Architektur und Funktionsweise des CryptoService .....     | 7   |
| 3.1. Aufbau.....                                              | 7   |
| 3.2. Komponenten .....                                        | 8   |
| 3.3. Ablauf.....                                              | 11  |
| 3.4. SCPs .....                                               | 12  |
| 3.5. Dokumentation .....                                      | 14  |
| 4. Umsetzung ausgewählter Secure Coding Practices .....       | 15  |
| 4.1. Input Validation.....                                    | 15  |
| 4.2. Access Control .....                                     | 18  |
| 4.3. Cryptographic Practices .....                            | 21  |
| 4.4. Error Handling and Logging.....                          | 23  |
| 4.5. Communication Security.....                              | 27  |
| 5. Bewertung und Reflexion .....                              | 30  |
| 5.1. Reflexion .....                                          | 30  |
| 5.2. Grenzen.....                                             | 30  |
| 5.3. Ausblick und mögliche Erweiterungen .....                | 31  |
| Literaturverzeichnis .....                                    | 32  |
| Anhang .....                                                  | 35  |
| Anhang A: Dokumentation der Verwendung von KI-Werkzeugen..... | 35  |

# Abbildungsverzeichnis

|                                                       |    |
|-------------------------------------------------------|----|
| Abbildung 1: Komponenten-Diagramm CryptoService ..... | 10 |
| Abbildung 2: Use-Case-Diagramm CryptoService .....    | 11 |

# 1. Einleitung

## 1.1. Motivation

### **Warum ist sichere Softwareentwicklung heute besonders relevant?**

Moderne Anwendungen steuern zunehmend kritische Bereiche wie Finanzsysteme, Medizintechnik oder Infrastrukturen, sodass unsichere Software gravierende Folgen haben kann, von wirtschaftlichem Schaden bis hin zur Gefährdung von Leben (IEEE, o.J.).

Angreifer nutzen hochentwickelte Techniken und suchen kontinuierlich nach Schwachstellen in Software-Systemen, was Sicherheitsmaßnahmen zur vorrangigen Aufgabe macht (Timbó, 2025).

Best Practices zeigen, dass Sicherheitsaspekte idealerweise bereits in der Planungsphase integriert werden sollten, da dadurch spätere Fehler und teure Nacharbeiten drastisch reduziert werden (IEEE, o.J.).

### **Was ist die praktische Relevanz von Secure Coding Practices (SCPs)?**

Secure Coding bedeutet, Sicherheitsprüfungen von Anfang an in den Entwicklungsprozess einzubinden, z. B. Input-Validierung, Output-Encoding oder sichere Authentifizierungstechniken (OWASP, o.J.). Studien zeigen, dass etwa 90 % aller Software-Schwachstellen auf fehlerhaftes Design oder ungesicherten Code zurückzuführen sind, sichere Kodierungsrichtlinien verringern dieses Risiko erheblich (Gasiba, et al., 2021). Die Implementierung von SCPs senkt signifikant die Wahrscheinlichkeit von Sicherheitsvorfällen und spart dadurch Zeit und Kosten im Betrieb (Sumanth, 2024).

### **Warum eignet sich ein CryptoService gut, um SCPs praxisnah zu demonstrieren?**

Ein CryptoService umfasst typische Komponenten wie Schlüsselverwaltung, Zufallsgeneratoren, Verschlüsselung sowie Integritätsprüfungen, alle Bereiche, in denen Secure Coding essenziell ist (OWASP, o.J.). Zudem lassen sich typische Schwachstellen wie unzureichende Validierung, falsches Error-Handling oder mangelhafte Autorisierung besonders gut simulieren und durch SCPs gezielt verhindern. Damit bietet ein CryptoService eine ideale Plattform, um Entwickler und Studierende praxisnah an Sicherheitsprinzipien heranzuführen und deren Wirkung sichtbar zu machen.

## 1.2. Zielsetzung

### **Was ist das Hauptziel dieser Projektarbeit?**

Das zentrale Ziel dieser Projektarbeit besteht darin, praxisrelevante SCPs systematisch zu identifizieren, umzusetzen und anhand eines konkreten Anwendungsszenarios, einem selbst entwickelten CryptoService, nachvollziehbar zu demonstrieren. Der Fokus liegt dabei auf

sicherheitskritischen Funktionen wie der Autorisierung, der Kryptographie, des Error-Handlings und Loggings sowie der Input-Validierung.

### **Was soll mit dem CryptoService demonstriert bzw. umgesetzt werden?**

Mit dem CryptoService soll exemplarisch gezeigt werden, wie sich sicherheitsrelevante Risiken durch konsequente Anwendung etablierter SCPs vermeiden oder minimieren lassen. Dazu werden gezielt potenzielle Schwachstellen in der Architektur, der Eingabeverarbeitung und der kryptografischen Logik adressiert.

### **Welche Rolle spielt die Verbindung zwischen Theorie (OWASP) und Praxis (eigene Implementierung)?**

Besonders im Vordergrund steht die Verbindung zwischen theoretischen Sicherheitsanforderungen, wie sie etwa von OWASP formuliert werden und deren praxisnaher Umsetzung in einem realitätsnahen Softwareprojekt. Ziel ist es, typische Anforderungen aus der Secure-Coding-Theorie konkret in Code zu überführen. Durch diese Verbindung von Theorie und Praxis soll ein vertieftes Verständnis für softwarebasierte Sicherheit gefördert werden.

## **1.3. Abgrenzung**

### **Welche Aspekte werden bewusst nicht behandelt?**

Diese Projektarbeit legt den Schwerpunkt bewusst auf die didaktische Vermittlung sicherheitsrelevanter Prinzipien anhand ausgewählter SCPs. Ziel ist nicht die vollständige Umsetzung aller in der Fachliteratur beschriebenen SCPs.

Ausdrücklich nicht Bestandteil der Arbeit sind Themen wie die Absicherung auf Systemebene oder eine vollständige Absicherung der gesamten Infrastruktur. Auch eine systematische Testabdeckung aller denkbaren Angriffsszenarien ist nicht vorgesehen. Die Entwicklung erfolgt ausschließlich lokal, ohne Deployment in produktionsnahen Umgebungen.

### **Welche Einschränkungen gelten für das Beispielsystem?**

Das Beispielsystem, der CryptoService, ist dabei bewusst als reduziertes Demonstrationssystem konzipiert. Ziel ist es nicht, ein marktfähiges Produkt zu entwickeln, sondern zentrale sicherheitsrelevante Herausforderungen exemplarisch nachvollziehbar zu machen.

### **Fokus: Didaktik und Prinzipienverständnis, nicht Full-Stack-Security**

Der Fokus liegt auf Prinzipienverständnis, nicht auf umfassender Full-Stack-Security. Die Arbeit verfolgt ein didaktisches Ziel. Leser sollen befähigt werden, grundlegende SCPs zu verstehen, zu bewerten und in eigenen Projekten anwenden zu können, nicht ein vollständiges Sicherheitssystem out-of-the-box zu übernehmen.

## 2. Grundlagen

### 2.1. Microservices

#### **Was versteht man unter einem Microservice? (Definition, Merkmale)**

Ein Microservice ist ein Bestandteil einer Softwarearchitektur, bei dem eine Anwendung in mehrere, unabhängig laufende Dienste zerlegt wird. Jeder Dienst läuft in einem eigenen Prozess und kommuniziert über leichte Schnittstellen (Amazon Web Services, Inc., o.J.) (Lewis & Fowler, 2014). Typisch ist, dass jeder Microservice eine klar abgegrenzte geschäftliche Funktion oder Bounded Context erfüllt (Microsoft, 2025). Sie sind lose gekoppelt, autonom deploybar und erlauben so unabhängige Entwicklung, Skalierung und Wartung (Harris, o.J.).

#### **Welche Eigenschaften sollte ein Microservice im Idealfall aufweisen?**

Ein idealer Microservice hat folgende Eigenschaften:

- Er ist klein und fokussiert, verständlich und ersetzbar (Wikipedia, 2025) (Richardson, o.J.).
- Er bietet hohe Autonomie: eigenständige Codebasis, Deployment-Pipeline und Datenhaltung (Wikipedia, 2024).
- Er ist skaliert unabhängig nach Bedarf und erhöht so Effizienz und Resilienz (Civardi, 2025).
- Er ist widerstandsfähig gegenüber Fehlern: Ausfall eines Services beeinflusst nicht das Gesamtsystem (Wikipedia, 2025).
- Er unterstützt Polyglottie und dezentrale Datenhaltung (Peplinski, 2025) (Amazon Web Services, Inc., o.J.).

#### **Inwiefern erfüllt der CryptoService diese Eigenschaften?**

- Der CryptoService ist in sich abgeschlossen, implementiert eine einzelne geschäftliche Funktion, nämlich kryptografische Operationen, und ist damit fokussiert und überschaubar.
- Er verfügt über eine eigene Codebasis und besitzt eine eigene Daten-/Schlüsselspeicherung, damit ist er autonom deploybar und wartbar.
- Die im CryptoService gegebene Modularität erlaubt Skalierung und isolierte Fehlerbehandlung.
- Technologisch kann z. B. die Verschlüsselung unabhängig implementiert werden, was Polyglottie und dezentrale Steuerung unterstützt.

#### **Wo ist der CryptoService möglicherweise „größer“ als klassisch gedacht, und warum ist das dennoch vertretbar?**

Im klassischen Microservice-Kontext sollten Services sehr klein sein. Obwohl der CryptoService mehrere kryptografische Komponenten enthält und damit größer ist als ein typischer Microservice, ist dies für den Lern- und Demonstrationszweck vertretbar. Zur Demonstration von



SCPs braucht es einen ausreichend umfassenden Kontext, damit die Wechselwirkungen zwischen Komponenten sichtbar werden.

Zudem bleibt der Demonstrator dennoch klein genug, um von einem einzelnen Team innerhalb eines überschaubaren Zeitrahmens umgesetzt zu werden, was im Einklang mit den Grundprinzipien von Microservices steht.

## 2.2. Kryptografie und Datensicherheit

### Was sind zentrale Konzepte moderner Kryptografie?

Symmetrische Verschlüsselung nutzt denselben geheimen Schlüssel für Ver- und Entschlüsselung und gehört zu den am häufigsten eingesetzten Verfahren in modernen Anwendungen (Sharifi, 2025).

Der AES ist heute der de-facto Standard für symmetrische Blockverschlüsselung (PyCryptodome, o.J.).

Ein IV dient zur Randomisierung, um dasselbe Klartextpaket unterschiedlich verschlüsselt darzustellen (Kampanakis, et al., 2024) (PyCryptodome, o.J.).

Im Modus AES-GCM (Galois/Counter Mode) werden Verschlüsselung und Message Authentication in einer AEAD-Konstruktion kombiniert, Vertraulichkeit und Integrität werden somit gleichzeitig gewährleistet (Wikipedia, 2025).

Ein Keystore stellt die sichere Aufbewahrung von Schlüsseln sicher, idealerweise isoliert, verschlüsselt und nur erreichbar durch autorisierte Komponenten.

### Warum ist die sichere Schlüsselspeicherung ein zentrales Sicherheitsziel?

Schlüssel sind der essenzielle Schutzmechanismus. Wenn ein Angreifer Zugriff auf Schlüssel hat, wird jede Verschlüsselung unwirksam. Unsichere Speicherung erhöht die Gefahr von Kompromittierung erheblich.

AES-GCM verstärkt diese Anforderungen, etwa durch strikte Vorgabe eines einzigartigen IV je Schlüsselverwendung (Kampanakis, et al., 2024).

### Was ist ein JWT, wie funktioniert es, und wie hilft es bei sicherem Zugriff?

Ein JWT (JSON Web Token) ist ein standardisiertes Tokenformat (RFC 7519), das Claims (z. B. Identität, Berechtigungen) in einem JSON-Payload enthält und mittels Signatur (z. B. HMAC-SHA256) verifiziert wird (Ibrahim, 2024).

JWTs können optional verschlüsselt werden oder nur signiert sein, signierte JWTs sind lesbar aber unveränderbar (Ibrahim, 2024).

### **Welche Sicherheitsziele werden durch den CryptoService unterstützt?**

Der CryptoService gewährleistet durch AES-GCM Vertraulichkeit und Integrität der Daten sowie durch JWT-Signaturen die Authentizität. Der CryptoService realisiert somit kryptografisch fundierte Mechanismen, die gezielt Vertraulichkeit, Integrität und Authentizität fördern, im Einklang mit modernen Sicherheitsprinzipien der Datensicherheit.

### **Was ist mTLS?**

Mutual TLS (mTLS) ist eine Erweiterung des klassischen TLS-Protokolls, bei der sich sowohl Client als auch Server gegenseitig mithilfe von X.509-Zertifikaten authentifizieren (Cloudflare, Inc., o.J.). Im Gegensatz zu einseitigem TLS, bei dem nur der Server authentifiziert wird, erfolgt bei mTLS eine zweiseitige Authentisierung (Verhelst, 2025).

mTLS wird häufig in Zero-Trust-Architekturen, im Bereich der Microservices-Kommunikation, bei APIs oder IoT-Geräten eingesetzt, wo beide Kommunikationspartner authentifiziert werden müssen (EJBCA by Keyfactor, 2023). Die Nutzung von mTLS setzt eine Public-Key-Infrastruktur (PKI) voraus, bei der Zertifikate entweder von einer internen oder externen Zertifizierungsstelle (CA) ausgestellt und überprüft werden (Verhelst, 2025). Dadurch gewährleistet mTLS in einer sicheren, verschlüsselten Verbindung gleichzeitig Vertraulichkeit, Integrität und beidseitige Authentizität des Kommunikationspartners (Cloudflare, Inc., o.J.).

## **2.3. OWASP SCPs**

### **Was sind SCPs laut OWASP?**

Die SCPs der OWASP sind eine technologieunabhängige Sammlung allgemeiner sicherheitsorientierter Programmierpraktiken. Es handelt sich um eine Art Leitfaden, der systematisch hilft, sichere Anwendungen zu bauen (OWASP, 2010). Unter folgendem Link stehen die verschiedenen Versionen des „OWASP Secure Coding Practices-Quick Reference Guide“ zum Download bereit: <https://owasp.org/www-project-secure-coding-practices-quick-reference-guide/>

### **Wie ist der „OWASP Secure Coding Practices-Quick Reference Guide“ von OWASP strukturiert?**

Der „OWASP Secure Coding Practices-Quick Reference Guide“ ist im Checklistenformat mit 14 thematischen Blöcken aufgebaut. Für diese Arbeit relevant sind die folgenden:

- Input validation
- Access control
- Cryptographic practices
- Error handling & logging
- Communication security

**Warum ist es nicht realistisch, alle SCPs gleichzeitig umzusetzen?**

OWASP selbst betont, dass das Dokument nicht im Detail beschreibt, wie jede Maßnahme implementiert wird, es gibt keine universelle Lösung für alle Praktiken (OWASP, 2010). Die Auswahl der anzuwendenden SCPs hängt von Projektzielen und Ressourcen ab.

Praktisch ist es oft nicht umsetzbar, alle SCP-Bereiche parallel vollständig zu implementieren, vor allem nicht in kleineren Projekten mit begrenzter Zeit und Ressourcen. Außerdem gibt es SCPs die sich auf einen spezifischen Kontext beziehen und diese somit in manchen Szenarien gar nicht anwendbar sind.

**Nach welchen Kriterien wurden die ausgewählten SCPs für diese Arbeit bestimmt?**

Relevanz: Fokus auf SCPs, die unmittelbar die Sicherheit kryptografischer Operationen betreffen.

Didaktik: Auswahl von SCPs mit sichtbarem Effekt im Demonstrationssystem.

Machbarkeit: Umsetzung nur der realistisch implementierbaren SCPs.

## 3. Architektur und Funktionsweise des CryptoService

### 3.1. Aufbau

Der CryptoService ist ein modular strukturierter Microservice, der grundlegende kryptografische Operationen sicher und nachvollziehbar umsetzt. Ziel des Dienstes ist es, Clients eine sichere Schnittstelle zur Verfügung zu stellen, über die kryptografische Schlüssel generiert, Nachrichten verschlüsselt bzw. entschlüsselt sowie JSON Web Tokens (JWTs) zur Nachrichtenidentifikation erzeugt werden können. Dabei folgt der Aufbau konsequent einem mehrschichtigen Architekturprinzip, das sowohl Wartbarkeit als auch Sicherheit und Testbarkeit gewährleistet.

Das Repository des Projektes steht unter folgendem Link zur Verfügung:  
<https://github.com/Marcey1611/Crypto-Microservice/tree/project-work>

#### **Schichtenarchitektur**

Der CryptoService folgt einer klar definierten Layer-Struktur:

- Boundary Layer: Beinhaltet die API, Validierung, Autorisierung, Security-Layer und den Controller.
- Facade Layer: Dient als Vermittlungsschicht, die Anfragen aus dem Boundary Layer an die Geschäftslogik delegiert und abstrahiert.
- Business Logic Layer: Implementiert die Kernfunktionalitäten des Key-Managements, der JWT-Verwaltung sowie der Kryptographie.
- Error Handling: Querschnittskomponente für die zentrale Behandlung und Protokollierung von Fehlern unter Beachtung sicherheitsrelevanter Aspekte.

#### **Berücksichtigung von Clean Code und Modularisierung**

Die Codebasis des Services ist unter Berücksichtigung etablierter Clean-Code-Prinzipien und Modularisierung umgesetzt, welche unter anderem folgende sind:

- Methoden sind kurz und auf eine klar definierte Aufgabe beschränkt.
- Klassen erfüllen das Single-Responsibility-Principle und sind nicht zu umfangreich.
- Wiederverwendbarkeit wurde durch Auslagerung von Funktionen realisiert.
- Redundanter Code wird vermieden, um Wartbarkeit/Übersichtlichkeit zu maximieren.

#### **CryptoClient**

Zur Demonstration und Testung der Funktionalität wurde ein sogenanntes CryptoClient-Projekt entwickelt, mit welchem die Kommunikation von Clients mit dem CryptoService über mTLS abgebildet werden kann. Die Clients generieren jeweils eigene Schlüssel, tauschen JWTs aus und kommunizieren über die API des CryptoService verschlüsselt miteinander. Dieses Setup erlaubt realitätsnahe Integrationstests unter produktionsähnlichen Bedingungen.

## **TLS-Absicherung**

Die gesamte Kommunikation zwischen einem Client und dem Service ist durch mTLS abgesichert. Dabei kommen Zertifikate zum Einsatz, die die Authentizität der beteiligten Parteien sicherstellen. Die dazugehörigen Zertifikate und Konfigurationsdateien für zwei Clients sind bereits erstellt. Eine vollständige Dokumentation zur Erstellung neuer Zertifikate sowie zur Struktur der TLS-Dateien findet sich in der Datei `how-to-create-mtls-files.md`, welche als Leitfaden zur mTLS-Integration dient.

## **3.2. Komponenten**

Der CryptoService ist in klar abgegrenzte Komponenten unterteilt, die jeweils für spezifische Sicherheits-, Verarbeitungs- oder Verwaltungsaufgaben verantwortlich sind. Diese modulare Struktur erhöht die Wartbarkeit, Testbarkeit und Nachvollziehbarkeit des Systems und folgt dem Prinzip der Separation of Concerns. Die wichtigsten Komponenten des CryptoService werden im Folgenden vorgestellt.

### **Security Layer**

Der Security Layer stellt die erste Verteidigungslinie des Dienstes dar und besteht unter Anderem aus den folgenden Filtern:

- `IpRateLimitingFilter` und `PrincipalRateLimitingFilter`: Begrenzt die Anzahl eingehender Requests pro IP-Adresse und Client-Zertifikat (Common Name, CN), um Missbrauch oder Brute-Force-Angriffe zu verhindern.
- `X509AuthenticationFilter`: Validiert die mTLS-Zertifikate eingehender Anfragen und extrahiert den CN zur Identifikation des Clients.

Die Filter agieren frühzeitig in der Verarbeitungskette und sichern somit den Zugriff bereits zu einem frühen Zeitpunkt ab.

### **Validation-Komponente**

Unter einem Zero-Trust-Paradigma werden alle Eingaben, einschließlich des aus dem Zertifikat extrahierten CN, strikt validiert. Hierzu zählen:

- Prüfung auf Null- oder Leereinträge,
- maximale Länge,
- erlaubte Zeichensätze (ASCII, UTF-8),
- Whitelist-Checks sowie
- Prüfung auf gültige und valide JWTs.

Diese Validierungen bilden eine zentrale Verteidigung gegen Angriffe wie Injection, Token-Manipulation oder Missbrauch von Schlüsselreferenzen.

## **Authorization**

Neben der mTLS-basierten Authentifizierung erfolgt eine fein granulare Autorisierung:

- Abhängig vom jeweiligen Interface (z. B. /encrypt, /decrypt) werden zusätzliche Autorisierungsprüfungen durchgeführt.
- Die Komponente AuthRegistry verwaltet dabei, welcher Client auf welchen keyAlias zugreifen darf.
- Der Zugriff wird per expliziter Positivliste gewährt, ein unbekannter oder nicht autorisierter Zugriff wird standardmäßig verweigert.

## **Key-Management**

Die Komponente für das Schlüsselmanagement übernimmt:

- die sichere Generierung symmetrischer Client-Schlüssel,
- die Verschlüsselung und Speicherung dieser Schlüssel mit einem serverseitig gehaltenen Master Key,
- die Verwaltung einer ClientKeyRegistry, in der gespeichert ist, welcher keyAlias zu welchem Client (CN) gehört.

Der Master Key selbst wird regelmäßig im Rahmen einer automatisierten Rotation erneuert. Zusätzlich entfernt ein Cleanup-Task regelmäßig abgelaufene Client-Schlüssel, um Ressourcen freizugeben und Angriffsflächen zu reduzieren.

## **JWT-Management**

Diese Komponente stellt Funktionen bereit zur:

- Generierung sicher signierter JWTs, die den keyAlias und den berechtigten issuedTo-Client enthalten,
- Validierung und Extraktion von Claims aus eingehenden JWTs, etwa zur Autorisierungsprüfung bei /decrypt.

## **Crypto-Komponente**

Der zentrale Bestandteil der Ver- und Entschlüsselung:

- Verwendung des Algorithmus AES-GCM mit 256-Bit-Schlüsseln und IVs zur Gewährleistung von Vertraulichkeit und Integrität,
- Handhabung von Base64-Encoding zur sicheren Übertragung von Cipher-Texten,
- Entschlüsselung nur unter Einhaltung der Validierung und Autorisierung.

Die Crypto-Komponente ist eng verzahnt mit dem JWT- und Key-Management, um den passenden Schlüssel zur Verarbeitung zu ermitteln.

## Error Handling

Fehlerbehandlung erfolgt konsistent und sicher durch:

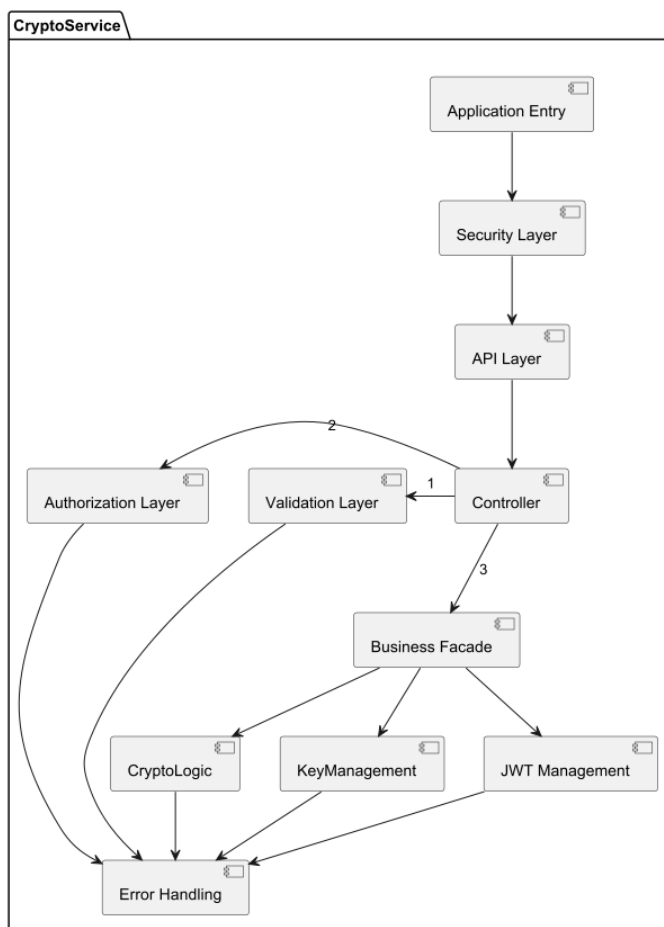
- eine zentrale ErrorHandler-Klasse,
- eine strukturierte Fehlerklassifikation mittels ErrorCode-Enum,
- gezielte Verwendung von Custom Exceptions (z. B. BadRequestException, ForbiddenException, InternalServerErrorException),
- Vermeidung technischer Details oder Stacktraces in API-Fehlermeldungen (gemäß OWASP-Empfehlung).

## Zusammenspiel der Komponenten

Die Komponenten sind funktional entkoppelt, interagieren jedoch gezielt:

- Die Crypto-Komponente ruft das JWT-Management auf, um Claims zu validieren und zu extrahieren.
- Der KeyStoreHelper ist eine zentrale Utility-Komponente, die von nahezu allen anderen Komponenten verwendet wird.
- Die Authorization und Validierung greifen auf das JWT-Management zurück, um sichere Entscheidungen zu treffen.

Abbildung 1: Komponenten-Diagramm CryptoService



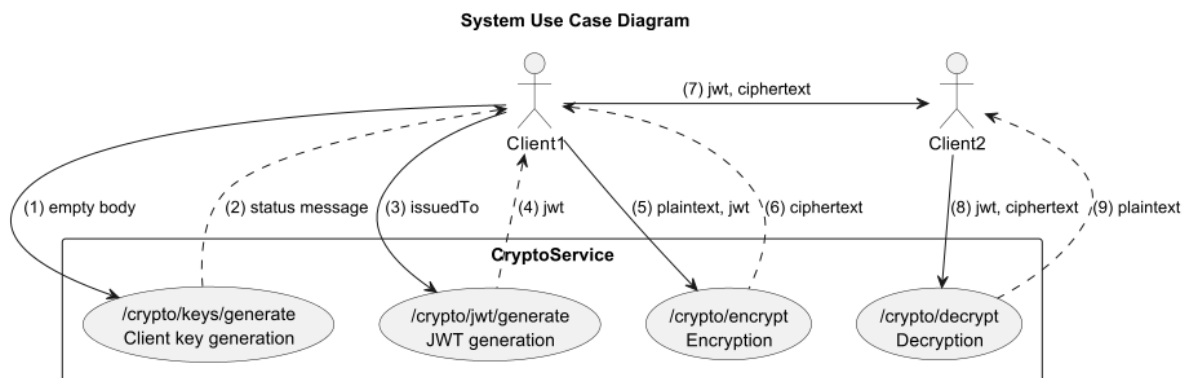
Eigene Abbildung

Das Diagramm verdeutlicht die Beziehungen und Aufrufhierarchien zwischen den Komponenten und kann als konzeptioneller Überblick dienen.

### 3.3. Ablauf

Der folgende Abschnitt beschreibt den vollständigen Ablauf einer typischen verschlüsselten Kommunikation zwischen zwei Clients über den CryptoService. Um die beteiligten Akteure und Interaktionen übersichtlich darzustellen, wird zunächst ein Use-Case-Diagramm verwendet.

Abbildung 2: Use-Case-Diagramm CryptoService



Eigene Darstellung

#### Szenario-Beschreibung

Im folgenden Anwendungsfall möchte Client1 eine Nachricht an Client2 verschlüsselt und sicher übermitteln. Der Ablauf gliedert sich in mehrere klar abgegrenzten Phasen:

##### 1. Key-Generierung durch Client1

Client1 initiiert die Kommunikation mit dem Service, indem er das Interface POST /keys/generate aufruft (1). Der Service generiert daraufhin einen symmetrischen AES-Schlüssel, speichert diesen intern verschlüsselt und weist ihm einen eindeutigen keyAlias zu. Die Antwort enthält keinen Schlüssel, sondern lediglich eine Bestätigung über den Erfolg oder Misserfolg der Generierung (2).

##### 2. JWT-Erstellung für die Nachrichtenzustellung an Client2

Im nächsten Schritt ruft Client1 das Interface POST /jwt/generate auf und übermittelt darin den Namen von Client2 (Common Name aus dem TLS-Zertifikat) als issuedTo (3). Der Service erstellt daraufhin einen signierten JWT, der folgende Claims enthält:

- keyAlias: Identifiziert den Schlüssel von Client1.
- issuedTo: Gibt an, dass dieser Token ausschließlich von Client2 genutzt werden darf.

Der JWT wird an Client1 zurückgegeben (4).



### 3. Nachricht verschlüsseln

Client1 ruft nun das Interface POST /crypto/encrypt auf und übergibt:

- den zu verschlüsselnden Plain-Text,
- den zuvor erhaltenen JWT (5).

Der Service überprüft die Gültigkeit des Tokens, validiert die Anfrage und verschlüsselt die Nachricht mit dem zugeordneten Client-Schlüssel. Als Antwort erhält Client1 den Cipher-Text, base64-kodiert (6).

### 4. Übermittlung an Client2

Der JWT und der verschlüsselte Cipher-Text werden von Client1 an Client2 übermittelt (7).

### 5. Nachricht entschlüsseln durch Client2

Client2 ruft das Interface POST /crypto/decrypt auf und übermittelt:

- den empfangenen JWT und
- den empfangenen Cipher-Text (8).

Der Service prüft die Signatur und Gültigkeit des JWTs, insbesondere ob issuedTo mit dem CN des anfragenden Clients (aus dem mTLS-Zertifikat) übereinstimmt. Der Cipher-Text wird anschließend mit dem ursprünglich verwendeten Schlüssel entschlüsselt, sofern alle Bedingungen erfüllt sind. Die Antwort enthält den wiederhergestellten Plain-Text (9).

## Zusammenfassung

Der CryptoService bildet mit diesem Ablauf ein durchgängig abgesichertes Kommunikationsmuster ab:

- Schlüssel bleiben ausschließlich auf Serverseite gespeichert.
- JWTs ermöglichen eine temporäre, kontrollierte Schlüsselreferenzierung.
- mTLS stellt sicher, dass nur authentifizierte Clients kommunizieren.
- Validierung und Autorisierung erfolgen mehrfach über alle Stufen hinweg.

Diese Architektur erlaubt es, Nachrichten zwischen beliebigen registrierten Clients sicher zu verschlüsseln und zu übertragen, ohne dass private Schlüssel jemals den Server verlassen.

## 3.4. SCPs

Der CryptoService wurde unter konsequenter Berücksichtigung ausgewählter SCPs gemäß den Empfehlungen der OWASP umgesetzt. Ziel war es, ein praxisnahes Sicherheitsniveau zu erreichen, ohne den Entwicklungsaufwand unnötig zu verkomplizieren. Der Fokus lag dabei auf fünf besonders relevanten Themenblöcken, die sich eng an den Risiken orientieren, denen kryptografische Dienste typischerweise ausgesetzt sind.

## Thematische Schwerpunkte

Die umgesetzten SCPs lassen sich in folgende Kategorien einteilen:

- Input Validation

Alle eingehenden Nutzereingaben, darunter auch sicherheitskritische Daten wie der Common Name (CN) aus dem mTLS-Zertifikat, JWTs oder freie Texteingaben werden umfassend validiert. Es kommen u. a. Whitelists, Längenprüfungen, Zeichensatz-Filter und strukturierte Regex-Prüfungen zum Einsatz. Diese Maßnahmen folgen einem strikten Zero-Trust-Modell und sollen potenzielle Angriffsvektoren wie Injection, Token Spoofing oder Unicode-Manipulation frühzeitig unterbinden.

- Access Control

Die Autorisierung erfolgt nach dem Prinzip der expliziten Positivliste. Jeder Zugriff auf kryptografische Schlüssel wird geprüft, nicht nur durch die Authentifizierung via mTLS, sondern auch über zusätzliche Prüfungen, etwa anhand des issuedTo-Claims in JWTs. Die Komponente AuthRegistry übernimmt die Zuordnung von Clients zu keyAliases und sorgt dafür, dass nur berechtigte Teilnehmer auf bestimmte Schlüssel zugreifen dürfen.

- Cryptographic Practices

Die gesamte kryptografische Verarbeitung folgt etablierten Best Practices:

- Verwendung von AES-GCM zur Sicherstellung von Vertraulichkeit und Integrität,
- sichere Generierung und Speicherung von Client-Schlüsseln,
- Master Key Rotation zur Begrenzung der Lebensdauer kryptografischer Materialien,
- Base64-Encoding sensibler Daten zur sicheren Übertragung,
- konsequente Trennung von Schlüsseln und deren Referenzierung über keyAliases.

- Error Handling & Logging

Fehler werden zentral behandelt und nicht an der Oberfläche preisgegeben. Die API liefert ausschließlich generische Fehlermeldungen, ohne technische Details oder Stacktraces offenzulegen.

Intern erfolgt die Protokollierung über eine zentrale ErrorHandler-Komponente und ein typisiertes ErrorCode-Enum. Erfolgs- und Sicherheitsereignisse werden selektiv und bewusst geloggt, insbesondere ohne kryptografisch sensitive Informationen wie Schlüssel, keyAliases oder vollständige JWTs.

- Communication Security

Die gesamte Kommunikation mit dem Dienst erfolgt über mTLS. Dadurch wird sichergestellt, dass nur authentifizierte Clients überhaupt mit dem CryptoService interagieren können.

Die TLS-Absicherung ist dokumentiert und reproduzierbar umgesetzt (siehe how-to-create-mtls-files.md). Zertifikate für neue Clients können anhand des vorhandenen Leitfadens sicher erstellt und eingebunden werden.

## **Dokumentation der SCP-Umsetzung im Code**

Die Umsetzung sicherheitsrelevanter Maßnahmen ist im Quellcode explizit nachvollziehbar gemacht. Wenn eine Klasse (oder Datei) zur Einhaltung eines bestimmten SCPs beiträgt, ist dies direkt im Javadoc-Kommentar am Anfang der Klasse dokumentiert.

Dies gewährleistet sowohl Transparenz als auch Nachvollziehbarkeit der Sicherheitsarchitektur im Rahmen von Code-Reviews, Penetration Tests oder Audits.

## **3.5. Dokumentation**

Um die Nachvollziehbarkeit, Wartbarkeit und Weiterverwendung des CryptoService zu unterstützen, wurde großer Wert auf eine strukturierte, nachvollziehbare und entwicklerfreundliche Dokumentation gelegt. Diese umfasst sowohl technische als auch konzeptionelle Aspekte des Systems und steht vollständig im Projekt-Repository zur Verfügung.

Im Verzeichnis „/documentation“ befindet sich ein Überblick über die Dokumentation („documentation/documentation-overview.md“). Diese Datei gibt einen Überblick über alle Teile der Dokumentation mit direkter Verlinkung.

### **Zielgruppe und Verwendungszweck**

Die Dokumentation richtet sich gleichermaßen an:

- Studierende oder Dritte, die den Service zu Lern- oder Demonstrationszwecken verwenden und
- Entwickler/Studierende, die den Service erweitern oder anpassen möchten.

Dank dieser strukturierten Dokumentation wird eine schnelle Einarbeitung und ein fundiertes Verständnis des Services in verschiedenen Kontexten ermöglicht.

## 4. Umsetzung ausgewählter Secure Coding Practices

### 4.1. Input Validation

#### **Conduct all data validation on a trusted system (e.g., The server) [1]**

Datenvalidierung muss auf der Serverseite erfolgen, da nur der Server als vertrauenswürdig gilt. Clientseitige Validierung allein reicht nicht aus und kann leicht umgangen werden.

Im CryptoService erfolgt jede Form der Eingabevalidierung ausschließlich auf dem Server. Die zentrale Komponente dafür ist der `ValidationService`, welcher systematisch alle eingehenden Nutzereingaben überprüft, unabhängig davon, ob sie von Client A oder B stammen. Dieses Vorgehen folgt einem Zero-Trust-Prinzip, bei dem grundsätzlich keiner externen Eingabe vertraut wird.

Link zum `ValidationService` im GitHub-Repository:

<https://github.com/Marcey1611/Crypto-Microservice/blob/project-work/cryptoservice/src/main/java/com/projectwork/cryptoservice/boundary/validation/ValidationService.java>

#### **Identify all data sources and classify them into trusted and untrusted. Validate all data from untrusted sources [2]**

Alle externen Daten müssen als potenziell unsicher gelten, bis sie validiert wurden. Nur intern erzeugte und kontrollierte Daten dürfen ohne zusätzliche Prüfungen verwendet werden.

Im CryptoService werden alle Daten, die von außen über die API kommen, grundsätzlich als untrusted behandelt – egal ob JWTs, Klartexte oder Parameter. Erst nach erfolgreicher Validierung durch den `ValidationService` werden sie weiterverarbeitet. Im Gegensatz dazu werden intern verwaltete Daten wie Key-Aliase, Schlüssel oder IVs, die im Serverkontext erzeugt und gespeichert werden, als trusted klassifiziert und bedürfen keiner erneuten Validierung. Dieses Konzept ist Teil des Zero-Trust-Modells des Services.

#### **There should be a centralized input validation routine for the application [3]**

Eingaben sollten nicht an mehreren Stellen dezentral validiert werden, sondern über eine zentrale, einheitliche Validierungslogik erfolgen.

Die gesamte Eingabevalidierung ist im CryptoService zentral über den `ValidationService` organisiert. Es existieren keine verstreuten Validierungsaufrufe im Code, sondern alle Prüfungen laufen über diesen Service, der wiederum gezielt spezialisierte Einzel-Validatoren (z. B. für ASCII, Steuerzeichen, Länge etc.) aufruft. Direkte Nutzung der Einzelvalidatoren außerhalb des `ValidationService` ist nicht vorgesehen, was eine klare Trennung und Wartbarkeit gewährleistet.

Link zum Package `validation` im GitHub-Repository: <https://github.com/Marcey1611/Crypto-Microservice/tree/project-work/cryptoservice/src/main/java/com/projectwork/cryptoservice/boundary/validation>

### **Specify proper character sets, such as UTF-8, for all sources of input [4]**

Für alle Eingaben sollte ein klar definierter Zeichensatz wie UTF-8 verwendet werden, um Inkonsistenzen und Interpretationsfehler zu vermeiden.

Im CryptoService wird der Zeichensatz standardmäßig durch die Verwendung von `@RequestBody` in den API-Interfaces auf UTF-8 gesetzt. Zusätzlich wurde ein dedizierter `CharsetValidator` integriert, der sicherstellt, dass sämtliche Eingaben explizit UTF-8-konform sind. Dadurch wird sichergestellt, dass nur gültige UTF-8-Daten verarbeitet werden, auch bei manipulierten Requests.

Link zum `CharsetValidator` im GitHub-Repository: <https://github.com/Marcey1611/Crypto-Microservice/blob/project-work/cryptoservice/src/main/java/com/projectwork/cryptoservice/boundary/validation/rule/CharsetValidator.java>

### **All validation failures should result in input rejection [6]**

Sobald eine Validierung fehlschlägt, muss die Verarbeitung abgebrochen und die Eingabe abgelehnt werden, ohne Ausnahme.

Im CryptoService führen alle Validierungsfehler konsequent zu einem Abbruch der Verarbeitung. Die einzelnen Validatoren im Paket `validation.rule` melden Fehler direkt an den `ErrorHandler`, der einen HTTP 400 Bad Request zurückgibt. Dadurch wird sichergestellt, dass fehlerhafte oder potenziell gefährliche Eingaben keine Auswirkungen auf die weitere Verarbeitung haben.

Link zum Package `validation.rule` mit den Validatoren im GitHub-Repository: <https://github.com/Marcey1611/Crypto-Microservice/tree/project-work/cryptoservice/src/main/java/com/projectwork/cryptoservice/boundary/validation/rule>

### **Validate all client-provided data before processing [8]**

Sämtliche Eingaben vom Client, egal aus welcher Quelle, müssen vollständig validiert werden, bevor sie weiterverarbeitet werden dürfen.

Im CryptoService wird jede clientseitige Eingabe, von einfachen Parametern bis hin zu den Claims innerhalb eines JWTs, vor jeglicher Weiterverarbeitung validiert. Diese Validierung erfolgt zentral über den `ValidationService` und folgt strikt dem Zero-Trust-Modell. Erst nach erfolgreicher Prüfung wird die Anfrage zur weiteren Verarbeitung weitergegeben.

Link zum `Controller` im GitHub-Repository, wo die Validierung für alle client-provided Daten aufgerufen wird: <https://github.com/Marcey1611/Crypto-Microservice/blob/project-work/cryptoservice/src/main/java/com/projectwork/cryptoservice/boundary/Controller.java>

### **Validate data length [13]**

Die Länge von Eingabedaten muss überprüft werden, um Buffer Overflows, DoS-Angriffe oder fehlerhafte Verarbeitungen zu verhindern.

Im CryptoService wird die maximale erlaubte Länge aller relevanten Felder durch den `LengthValidator` geprüft. Zusätzlich stellt der `NullOrBlankValidator` sicher, dass keine leeren Eingaben verarbeitet werden. Beide Validatoren werden zentral über den `ValidationService` aufgerufen und verhindern so unzulässige oder schadhafte Eingaben.

Link zum `LengthValidator` im GitHub-Repository: <https://github.com/Marcey1611/Crypto-Microservice/blob/project-work/cryptoservice/src/main/java/com/projectwork/cryptoservice/boundary/validation/rule/LengthValidator.java>

Link zum `NullOrBlankValidator` im GitHub-Repository: <https://github.com/Marcey1611/Crypto-Microservice/blob/project-work/cryptoservice/src/main/java/com/projectwork/cryptoservice/boundary/validation/rule/NullOrBlankValidator.java>

### **Validate all input against a "white" list of allowed characters, whenever possible [14]**

Statt nur gefährliche Zeichen zu blockieren, sollten nur ausdrücklich erlaubte Zeichen akzeptiert werden (Whitelist-Prinzip).

Im CryptoService wird bei nahezu allen Eingaben eine Whitelist-Prüfung durch den `WhitelistValidator` durchgeführt. Dieser erlaubt nur vorab definierte, sichere Zeichen. Für Felder wie den Ciphertext, der Sonderzeichen enthalten darf, wird alternativ eine Base64-Prüfung verwendet, um sicherzustellen, dass der Inhalt den erwarteten Zeichensatz erfüllt.

Link zum `WhitelistValidator` im GitHub-Repository: <https://github.com/Marcey1611/Crypto-Microservice/blob/project-work/cryptoservice/src/main/java/com/projectwork/cryptoservice/boundary/validation/rule/WhitelistValidator.java>

Link zum `Base64Validator` im GitHub-Repository: <https://github.com/Marcey1611/Crypto-Microservice/blob/project-work/cryptoservice/src/main/java/com/projectwork/cryptoservice/boundary/validation/rule/Base64Validator.java>

### **Discrete checks for null bytes, newline chars, path traversal [16]**

Eingaben sollten gezielt auf kontrollkritische Zeichen wie Null-Bytes (`\0`), Zeilenumbrüche (`\n`) oder Pfadmanipulationen überprüft werden.

Im CryptoService übernimmt der `ControlCharValidator` die dedizierte Prüfung auf Null-Bytes, Zeilenumbrüche und andere Steuerzeichen, die zu Sicherheitsrisiken führen können. Eine Pfadmanipulationsprüfung ist nicht erforderlich, da Eingaben an keiner Stelle zur Pfaderzeugung oder Dateiverarbeitung verwendet werden.

Link zum `ControlCharValidator` im GitHub-Repository: <https://github.com/Marcey1611/Crypto-Microservice/blob/project-work/cryptoservice/src/main/java/com/projectwork/cryptoservice/boundary/validation/rule/ControlCharValidator.java>

## 4.2. Access Control

### **Use only trusted system objects (e.g. server-side session objects) for making access authorization decisions [77]**

Entscheidungen über Zugriffe dürfen nur auf Basis von vertrauenswürdigen, serverseitig validierten Informationen getroffen werden.

Im CryptoService basieren alle Zugriffsentscheidungen auf validierten, serverseitig kontrollierten Objekten. Dazu gehören:

- Der `keyAlias` und `issuedTo` im JWT, dessen Signatur durch einen sicheren, serverseitigen Signing Key geprüft wird,
- sowie der Common Name (CN) aus dem mTLS-Zertifikat.

Diese Informationen werden im `AuthService` ausgewertet, um etwa im `ClientRegistry` den Zugriff auf `/encrypt` und `/decrypt` gezielt zu erlauben oder zu verweigern, nach erfolgreicher Validierung.

Link zum Controller im GitHub-Repository, hier ist zu sehen wie die Daten erst validiert werden und dann dem `AuthService` übergeben werden (beispielsweise Zeile 82-86):

<https://github.com/Marcey1611/Crypto-Microservice/blob/project-work/cryptoservice/src/main/java/com/projectwork/cryptoservice/boundary/Controller.java>

Link zum `AuthService` im GitHub-Repository: <https://github.com/Marcey1611/Crypto-Microservice/blob/project-work/cryptoservice/src/main/java/com/projectwork/cryptoservice/boundary/authorization/AuthService.java>

### **Use a single site-wide component to check access authorization [78]**

Zugriffsentscheidungen sollten nicht verteilt im Code, sondern über eine zentrale Autorisierungskomponente getroffen werden.

Im CryptoService übernimmt der `AuthService` alle Autorisierungsentscheidungen zentral. Es gibt keine verteilte oder doppelte Zugriffslogik, alle sicherheitskritischen Prüfungen (z. B. ob ein Client Zugriff auf einen Schlüssel hat) laufen gebündelt über diese eine Stelle. Das erhöht die Wartbarkeit und reduziert das Risiko inkonsistenter Sicherheitslogik.

Link zum `AuthService` im GitHub-Repository: <https://github.com/Marcey1611/Crypto-Microservice/blob/project-work/cryptoservice/src/main/java/com/projectwork/cryptoservice/boundary/authorization/AuthService.java>

### **Access controls should fail securely [79]**

Wenn ein Fehler bei der Zugriffskontrolle auftritt, muss der Zugriff standardmäßig verweigert werden, niemals erlaubt.

Im CryptoService schlagen alle Autorisierungsprüfungen im AuthService sicher fehl, d. h. bei fehlerhaften oder fehlenden Authentifizierungsdaten wird der Zugriff verweigert. Über das zentrale Error Handling wird dann ein entsprechender HTTP 401 (Unauthorized) oder 403 (Forbidden) Status-Code zurückgegeben. So wird sichergestellt, dass Zugriff nur bei eindeutig validierter Berechtigung gewährt wird.

Link zum AuthService im GitHub-Repository (beispielsweise Zeile 40-43 oder Zeile 59-62): <https://github.com/Marcey1611/Crypto-Microservice/blob/project-work/cryptoservice/src/main/java/com/projectwork/cryptoservice/boundary/authorization/AuthService.java>

### **Deny all access if the application cannot access its security configuration information [80]**

Wenn sicherheitsrelevante Konfigurationsdaten fehlen oder ungültig sind, muss der Zugriff vollständig verweigert und die Anwendung ggf. gestoppt werden.

Im CryptoService prüft der KeyStoreInitializer beim Start des Systems, ob der Keystore geladen werden kann und ob essenzielle Schlüssel wie der Master-Key und der JWT-Signing-Key vorhanden und gültig sind. Ist das nicht der Fall, wird die Anwendung kontrolliert abgebrochen und ein entsprechender Fehler geloggt.

Link zum KeyStoreInitializer im GitHub-Repository: <https://github.com/Marcey1611/Crypto-Microservice/blob/project-work/cryptoservice/src/main/java/com/projectwork/cryptoservice/businesslogic/keymanagement/KeyStoreInitializer.java>

### **Restrict access to services to only authorized users [87]**

Zugriff auf sicherheitsrelevante Dienste darf nur autorisierten Nutzern gewährt werden, öffentliche Schnittstellen sind zu vermeiden.

Im CryptoService sind alle sicherheitskritischen Endpunkte zugriffsbeschränkt:

- Auf den Encrypt- und Decrypt-Service kann nur zugegriffen werden, wenn eine gültige Autorisierung vorliegt.
- Für die JWT-Generierung muss der aufrufende Client zuvor im ClientKeyRegistry registriert worden sein.
- Beim Key-Generierungs-Endpunkt wird der Zugriff ausschließlich über mTLS abgesichert, d. h. nur Clients mit gültigem Zertifikat erhalten Zugriff.

Diese Regeln werden zentral über den AuthService durchgesetzt.

Link zum AuthService im GitHub-Repository: <https://github.com/Marcey1611/Crypto-Microservice/blob/project-work/cryptoservice/src/main/java/com/projectwork/cryptoservice/boundary/authorization/AuthService.java>



## **Restrict access to application data to only authorized users [88]**

Nur Nutzer mit gültiger Autorisierung dürfen auf interne Anwendungsdaten zugreifen, insbesondere bei sicherheitsrelevanten Ressourcen wie Schlüsseln.

Im CryptoService ist der Zugriff auf sensible Daten wie Keys und durch zentrale Autorisierungsprüfungen im AuthService geschützt.

- Beim Encrypt-, Decrypt- und JWT-Service darf ein Client nur auf die Daten zugreifen, für die er vorher autorisiert wurde.
- Beim Key-Generierungs-Endpunkt hingegen findet noch kein Zugriff auf bestehende Daten statt, sondern es wird ein neuer Schlüssel erstellt, daher ist dort keine zusätzliche Datenfreigabe erforderlich.

Die Zugriffsbeschränkung erfolgt konsistent über den AuthService.

Link zum AuthService im GitHub-Repository: <https://github.com/Marcey1611/Crypto-Microservice/blob/project-work/cryptoservice/src/main/java/com/projectwork/cryptoservice/boundary/authorization/AuthService.java>

## **Limit the number of transactions a single user/device can perform in a given time [94]**

Zur Vermeidung von Missbrauch sollten pro Nutzer oder Gerät nur begrenzte Anfragen innerhalb eines Zeitfensters erlaubt sein (Rate Limiting).

Der CryptoService setzt zweifaches Rate Limiting um:

- Der IpRateLimitingFilter begrenzt Anfragen basierend auf der IP-Adresse,
- der PrincipalRateLimitingFilter zusätzlich basierend auf dem Client-Zertifikat (CN aus dem mTLS Principal).

Diese Kombination erschwert eine Umgehung über IP-Wechsel und schützt sowohl vor automatisierten Angriffen als auch vor API-Missbrauch. Die Filter sind in der SecurityConfig global eingebunden und schützen alle relevanten Endpunkte.

Link zur SecurityConfig im GitHub-Repository: <https://github.com/Marcey1611/Crypto-Microservice/blob/project-work/cryptoservice/src/main/java/com/projectwork/cryptoservice/boundary/security/SecurityConfig.java>

Link zum IpRateLimitingFilter im GitHub-Repository: <https://github.com/Marcey1611/Crypto-Microservice/blob/project-work/cryptoservice/src/main/java/com/projectwork/cryptoservice/boundary/security/IpRateLimitingFilter.java>

Link zum PrincipalRateLimitingFilter im GitHub-Repository: <https://github.com/Marcey1611/Crypto-Microservice/blob/project-work/cryptoservice/src/main/java/com/projectwork/cryptoservice/boundary/security/PrincipalRateLimitingFilter.java>

### **Create an Access Control Policy documenting rules, data types, access logic [100]**

Es muss eine dokumentierte Zugriffskontrollrichtlinie existieren, die beschreibt, wer auf welche Daten unter welchen Bedingungen zugreifen darf.

Die Zugriffslogik des CryptoService wurde in einer eigenständigen Datei `access-control-policy-scp100.md` dokumentiert. Darin werden die Berechtigungsregeln für alle Endpunkte, die betroffenen Datenarten und die technische Zugriffskontrolle detailliert beschrieben.

Diese Dokumentation trägt zur Transparenz und Nachvollziehbarkeit der sicherheitsrelevanten Entscheidungen bei.

Link zur Access-Control-Policy im GitHub-Repository: <https://github.com/Marcey1611/Crypto-Microservice/blob/project-work/documentation/cryptoservice/access-control/access-control-policy-scp100.md>

## **4.3. Cryptographic Practices**

### **All cryptographic functions used to protect secrets from the application user must be implemented on a trusted system (e.g., the server) [101]**

Alle kryptografischen Operationen, insbesondere zur Schlüsselerwaltung und Datenverschlüsselung, müssen ausschließlich auf serverseitig kontrollierten Systemen erfolgen, nicht beim Nutzer selbst.

Im CryptoService werden sämtliche kryptografischen Funktionen ausschließlich auf dem Server ausgeführt:

- Die Schlüsselerzeugung und -verwaltung liegt vollständig beim Service,
- die Ver- und Entschlüsselung wird durch die Klassen `EncryptService` und `DecryptService` durchgeführt,
- die eigentlichen Schlüssel verlassen den Server niemals.

Das bedeutet, der Client übergibt lediglich Eingabedaten und erhält verschlüsselte oder entschlüsselte Ergebnisse zurück, ohne Zugriff auf die Kryptografie selbst. Dadurch bleibt die Kontrolle über die Schlüssel zu 100 % beim Server, was dem Prinzip eines Trusted Systems entspricht.

### **Protect master secrets from unauthorized access [102]**

Hauptschlüssel (z. B. Master Keys oder JWT-Signing Keys) müssen vor unbefugtem Zugriff streng geschützt werden, sowohl im Speicher als auch im Zugriffspfad.

Der CryptoService speichert sowohl den Master Key als auch den JWT-Signing Key sicher im Keystore, der nur über kontrollierte Methoden und mit dem richtigen Zugriffspasswort gelesen werden kann.

Dieses Passwort wird nicht im Code hinterlegt, sondern über eine Umgebungsvariable geladen. Zwar ließe sich dies mit einem dedizierten Secret-Management-Tool noch weiter absichern, aber

auch in der aktuellen Form ist kein direkter Zugriff auf die Schlüssel ohne explizite Berechtigung möglich.

Link zum KeyStoreLoader im GitHub-Repository: <https://github.com/Marcey1611/Crypto-Microservice/blob/project-work/cryptoservice/src/main/java/com/projectwork/cryptoservice/businesslogic/keymanagement/KeyStoreLoader.java>

### **Cryptographic modules should fail securely [103]**

Wenn ein Fehler in einem kryptografischen Modul auftritt, muss die Anwendung kontrolliert und sicher abbrechen, ohne sensible Informationen preiszugeben oder unsichere Zustände zu erzeugen.

Im CryptoService werden alle kryptografischen Fehler zentral und sicher behandelt. Tritt z. B. in CryptoUtility, EncryptService oder DecryptService eine Exception auf, wird sie nicht direkt weitergegeben, sondern durch ein custom Error Handling verarbeitet. Dieses nutzt ein vordefiniertes ErrorCode-Enum und gibt nur abstrakte, nicht sensible Fehlermeldungen an den Client weiter. Damit wird sichergestellt, dass der Service bei Kryptofehlern nicht in einen unsicheren Zustand übergeht und keine internen Details preisgibt.

Link zum EncryptService im GitHub-Repository (beispielsweise Zeile 116-122):  
<https://github.com/Marcey1611/Crypto-Microservice/blob/project-work/cryptoservice/src/main/java/com/projectwork/cryptoservice/businesslogic/cryptography/EncryptService.java>

Link zum DecryptService im GitHub-Repository (beispielsweise Zeile 167-173):  
<https://github.com/Marcey1611/Crypto-Microservice/blob/project-work/cryptoservice/src/main/java/com/projectwork/cryptoservice/businesslogic/cryptography/DecryptService.java>

Link zur CryptoUtility-Klasse im GitHub-Repository (beispielsweise Zeile 47-53):  
<https://github.com/Marcey1611/Crypto-Microservice/blob/project-work/cryptoservice/src/main/java/com/projectwork/cryptoservice/businesslogic/cryptography/CryptoUtility.java>

### **All random numbers, random file names, random GUIDs, and random strings should be generated using the cryptographic module's approved random number generator when these random values are intended to be un-guessable [104]**

Wenn Zufallswerte wie Schlüssel, IVs oder Dateinamen sicher sein müssen, müssen sie mit einem kryptografisch abgesicherten Zufallszahlengenerator erzeugt werden, einfache Methoden wie `Math.random()` sind ungeeignet.

Im CryptoService wird für alle sicherheitskritischen Zufallswerte, darunter:

- AES-Schlüssel,
- IVs und
- Key-Aliase,

ausschließlich `SecureRandom.getInstanceStrong()` verwendet. Diese Methode gilt als eine der sichersten verfügbaren Zufallsquellen in Java und ist für kryptografische Zwecke zertifiziert. Weder `Math.random()` noch `UUID.randomUUID()` werden verwendet, da diese nicht kryptografisch sicher sind.

Beispielsweise im `EncryptService` Zeile 142: <https://github.com/Marcey1611/Crypto-Microservice/blob/project-work/cryptoservice/src/main/java/com/projectwork/cryptoservice/businesslogic/cryptography/EncryptService.java>

Oder im `KeyManagementService` Zeile 92: <https://github.com/Marcey1611/Crypto-Microservice/blob/project-work/cryptoservice/src/main/java/com/projectwork/cryptoservice/businesslogic/keymanagement/KeyManagementService.java>

### **Establish and utilize a policy and process for how cryptographic keys will be managed [106]**

Für das Management kryptografischer Schlüssel muss es eine klare, dokumentierte Richtlinie und einen definierten Ablauf geben, von der Erzeugung bis zur Löschung.

Im `CryptoService` wurde eine eigene Datei `key-management-policy-scp106.md` erstellt, die den gesamten Lebenszyklus der Schlüssel beschreibt. Darin enthalten sind unter anderem:

- Regeln zur Erzeugung und Benennung von Schlüsseln (`KeyAlias`),
- Hinweise zur Speicherung im Keystore,
- sowie Vorgaben zur Rotation von Master-Keys (durch die `MasterKeyRotationTask`).

Diese Richtlinie sorgt für Konsistenz, Sicherheit und Nachvollziehbarkeit im Umgang mit kryptografischem Material.

Link zur Key-Management-Policy im GitHub-Repository:

<https://github.com/Marcey1611/Crypto-Microservice/blob/project-work/documentation/cryptoservice/key-management/key-management-policy-scp106.md>

## **4.4. Error Handling and Logging**

### **Do not disclose sensitive information in error responses [107]**

Fehlermeldungen dürfen keine internen oder sensiblen Details enthalten, um Angreifern keine zusätzlichen Informationen zu liefern.

Im `CryptoService` sind alle Fehlerantworten bewusst generisch formuliert, z. B. "Internal server error" oder "Invalid input". Die konkrete Ursache oder interne Details werden nicht an den Client weitergegeben. Stattdessen wird intern ein `ErrorCode`-Enum genutzt, um Fehler eindeutig zu identifizieren und zu loggen, ohne sicherheitsrelevante Informationen preiszugeben.

Link zum `ErrorCode`-Enum im GitHub-Repository: <https://github.com/Marcey1611/Crypto-Microservice/blob/project-work/cryptoservice/src/main/java/com/projectwork/cryptoservice/errorhandling/util/ErrorCode.java>

### **Use error handlers that do not display debugging or stack trace information [108]**

Fehlerantworten dürfen keine Debug-Ausgaben oder Stack-Traces enthalten, um keine internen Strukturen offenzulegen.

Im `CryptoService` sorgt der zentrale `ErrorHandler` dafür, dass keine Exceptions oder Stack-Traces an den Client weitergegeben werden. Selbst nicht explizit abgefangene Fehler werden durch den globalen Exception-Handler verarbeitet und führen zu einer generischen HTTP-500-Antwort (Internal Server Error). Intern werden die Fehler detailliert geloggt, aber nach außen nur in sicherer, abstrakter Form kommuniziert.

Link zum `ErrorHandler` im GitHub-Repository: <https://github.com/Marcey1611/Crypto-Microservice/blob/project-work/cryptoservice/src/main/java/com/projectwork/cryptoservice/errorhandling/util/ErrorHandler.java>

Link zum `GlobalExceptionHandler` im GitHub-Repository: <https://github.com/Marcey1611/Crypto-Microservice/blob/project-work/cryptoservice/src/main/java/com/projectwork/cryptoservice/errorhandling/GlobalExceptionHandler.java>

### **Implement generic error messages and use custom error pages [109]**

Fehlermeldungen sollten allgemein formuliert und nicht systemspezifisch sein. Zudem sollten bei Fehlern benutzerdefinierte Fehlerseiten oder -antworten angezeigt werden.

Der `CryptoService` gibt nur generische Fehlernachrichten aus, z. B. "Internal server error" oder "Invalid input", ohne technische Details preiszugeben. Diese Antworten werden zentral im `ErrorHandler` mithilfe des `ErrorCode`-Enums definiert. So ist sichergestellt, dass alle Fehler einheitlich und sicher dargestellt werden, unabhängig davon, wo sie im Code auftreten.

Link zum `ErrorCode`-Enum im GitHub-Repository: <https://github.com/Marcey1611/Crypto-Microservice/blob/project-work/cryptoservice/src/main/java/com/projectwork/cryptoservice/errorhandling/util/ErrorCode.java>

### **The application should handle application errors and not rely on the server configuration [110]**

Die Anwendung muss eigene Fehlerbehandlung implementieren und darf sich nicht auf Standardantworten des Servers verlassen.

Der `CryptoService` nutzt einen zentralen `ErrorHandler`, der Ausnahmen server-intern abfängt, in definierte `ErrorCode`-Antworten übersetzt und konsistente HTTP-Statuscodes

zurückgibt. So entstehen einheitliche, sichere Fehlerantworten, unabhängig von der Server-Defaultkonfiguration.

### **Error handling logic associated with security controls should deny access by default [112]**

Bei Unklarheiten oder Fehlern in sicherheitsrelevanten Prüfungen muss immer die verwehrende Entscheidung getroffen werden (deny-by-default).

Der CryptoService verweigert bei jedem Validierungs- oder Autorisierungsfehler den Zugriff:

- `ValidationService` → 400 Bad Request bei ungültigen Eingaben,
- `AuthService` → 401 Unauthorized (fehlende/ungültige Auth) bzw. 403 Forbidden (keine Berechtigung),
- Unerwartete Fehler in `KeyManagementService` & Co. → 500 Internal Server Error über den zentralen `ErrorHandler`.

Kryptografische Operationen werden in Fehlerfällen nicht ausgeführt, es gibt keine Teilverarbeitung.

### **All logging controls should be implemented on a trusted system (e.g., the server) [113]**

Sämtliche Logging-Mechanismen müssen serverseitig umgesetzt werden. Client- oder Browser-Logs sind nicht vertrauenswürdig.

Das gesamte Logging erfolgt ausschließlich auf dem Server. Sicherheitsrelevante Ereignisse und Validierungs-/Fehlerfälle werden zentral im errorhandling-Paket erfasst und über den `ErrorHandler` in sichere, konsistente Logeinträge übersetzt, ohne sensible Daten preiszugeben.

### **Logging controls should support both success and failure of specified security events [114]**

Bei definierten sicherheitsrelevanten Aktionen sollten sowohl erfolgreiche als auch fehlgeschlagene Ereignisse protokolliert werden.

Im CryptoService werden alle sicherheitskritischen Aktionen, darunter Verschlüsselung, Entschlüsselung, Validierung, Autorisierung, Schlüsselgenerierung sowie Laden und Speichern des Keystores, geloggt. Dabei werden nicht nur Fehlerfälle, sondern auch erfolgreiche Ausführungen erfasst.

Beispielsweise wird im `Controller` die erfolgreiche Validierung geloggt (beispielsweise Zeile 63): <https://github.com/Marcey1611/Crypto-Microservice/blob/project-work/cryptoservice/src/main/java/com/projectwork/cryptoservice/boundary/Controller.java>

Durch das Aufrufen des `ErrorHandler`s wenn eine Validierung fehlschlägt wird dies ebenfalls geloggt (Zeile 208-213 bzw. die Zeilen 44, 62, 78, 96 etc.): <https://github.com/Marcey1611/Crypto-Microservice/blob/project-work/cryptoservice/src/main/java/com/projectwork/cryptoservice/errorhandling/util/ErrorHandler.java>

### **Do not store sensitive information in logs [119]**

Logs dürfen keine vertraulichen Daten wie Passwörter, Schlüssel, geheime Pfade oder personenbezogene Informationen enthalten.

Im gesamten CryptoService werden keine sensiblen Daten geloggt. Clientnamen erscheinen nur einmal pro Request im Controller zu Debug-/Monitoringzwecken. KeyAliase, Passwörter, Pfade oder kryptografische Schlüssel werden nie in Logs ausgegeben. Damit wird sichergestellt, dass Logs keine Informationen enthalten, die zur Kompromittierung des Systems genutzt werden könnten.

### **Log all input validation failures [121]**

Jeder Fehler bei der Eingabevalidierung muss protokolliert werden, um Sicherheitsvorfälle nachvollziehen zu können.

Im CryptoService wird jeder Validierungsfehler automatisch geloggt. Die Validatoren im Paket `validation.rule` lösen bei fehlerhaften Eingaben den zentralen `ErrorHandler` aus. Dieser schreibt einen Logeintrag mit Fehlercode und beschreibt, welcher Validierungsschritt fehlgeschlagen ist.

Beispielsweise wird im `LengthValidator` bei zu langem Input in Zeile 35 der `ErrorHandler` aufgerufen: <https://github.com/Marcey1611/Crypto-Microservice/blob/project-work/cryptoservice/src/main/java/com/projectwork/cryptoservice/boundary/validation/rule/LengthValidator.java>

Der `ErrorHandler` logt jeden Fehler (Zeile 208-213 bzw. die Zeilen 44, 62, 78, 96 etc.): <https://github.com/Marcey1611/Crypto-Microservice/blob/project-work/cryptoservice/src/main/java/com/projectwork/cryptoservice/errorhandling/util/ErrorHandler.java>

### **Log all access control failures [123]**

Jeder fehlgeschlagene Zugriffskontrollversuch muss vollständig geloggt werden, um Sicherheitsvorfälle nachvollziehen zu können.

Im CryptoService wird jeder Fehler bei der Zugriffskontrolle, z. B. in `AuthService`, `IpRateLimitingFilter` oder `PrincipalRateLimitingFilter`, vom zentralen Error Handling erfasst und geloggt. Dabei werden Fehlercode, Kontext und Art des Fehlers protokolliert, ohne sensible Daten wie Schlüssel oder vollständige Identitäten im Log abzulegen.

Beispielsweise in Zeile 40-43 im `AuthService`: <https://github.com/Marcey1611/Crypto-Microservice/blob/project-work/cryptoservice/src/main/java/com/projectwork/cryptoservice/boundary/authorization/AuthService.java>

Der `ErrorHandler` logt jeden Fehler (Zeile 208-213 bzw. die Zeilen 44, 62, 78, 96 etc.): <https://github.com/Marcey1611/Crypto-Microservice/blob/project-work/cryptoservice/src/main/java/com/projectwork/cryptoservice/errorhandling/util/ErrorHandler.java>

### **Log all system exceptions [126]**

Jede aufgetretene Exception muss zentral geloggt werden, um Ursachenanalyse, Forensik und Monitoring zu ermöglichen.

Der `ErrorHandler` schreibt alle abgefangenen Exceptions mit `ErrorCode` und Kontext ins Log. Zusätzlich fängt der `GlobalExceptionHandler` sämtliche nicht explizit behandelten Ausnahmen ab, loggt sie und gibt eine standardisierte 500-Antwort zurück, ohne sensible Details in der Response.

Der `ErrorHandler` logt jeden Fehler in Zeile 208-213 bzw. in den Zeilen 44, 62, 78, 96 etc.:  
<https://github.com/Marcey1611/Crypto-Microservice/blob/project-work/cryptoservice/src/main/java/com/projectwork/cryptoservice/errorhandling/util/ErrorHandler.java>

### **Log cryptographic module failures [129]**

Jeder Fehler in einem kryptografischen Modul muss vollständig geloggt werden, um sicherheitsrelevante Vorfälle analysieren zu können.

Alle kryptografischen Operationen im `CryptoService`, z. B. in `CryptoUtility`, `EncryptService` und `DecryptService`, sind in try-catch-Blöcke eingebettet. Tritt eine Exception auf, wird sie über das zentrale Error Handling automatisch geloggt und in einen generischen, sicheren Fehlercode übersetzt. Dadurch werden alle Krypto-Fehler erfasst, ohne sensible Daten im Log zu hinterlegen.

Der `ErrorHandler` logt jeden Fehler in Zeile 208-213 bzw. in den Zeilen 44, 62, 78, 96 etc.:  
<https://github.com/Marcey1611/Crypto-Microservice/blob/project-work/cryptoservice/src/main/java/com/projectwork/cryptoservice/errorhandling/util/ErrorHandler.java>

## **4.5. Communication Security**

### **Implement encryption for the transmission of all sensitive information [143]**

Sämtliche sensiblen Daten müssen transportverschlüsselt übertragen werden (TLS), um Vertraulichkeit und Integrität auf dem Transportweg sicherzustellen.

Der `CryptoService` erzwingt TLS für alle Endpunkte. In der aktuellen Konfiguration ist mTLS aktiviert, sodass sich Client und Server gegenseitig per Zertifikat authentifizieren. Die Aktivierung und Parameter (Keystore/Truststore, Protokolle, Ports) sind in `SecurityConfig.java` und `application.properties` hinterlegt.

Link zur `application.properties` im GitHub-Repository (Zeile 9-14):  
<https://github.com/Marcey1611/Crypto-Microservice/blob/project-work/cryptoservice/src/main/resources/application.properties>



Link zur SecurityConfig im GitHub-Repository (siehe Zeile 55-69):

<https://github.com/Marcey1611/Crypto-Microservice/blob/project-work/cryptoservice/src/main/java/com/projectwork/cryptoservice/boundary/security/SecurityConfig.java>

**TLS certificates should be valid and have the correct domain name, not be expired, and be installed with intermediate certificates when required [144]**

TLS-Zertifikate müssen gültig (nicht abgelaufen) sein, den korrekten Hostnamen enthalten und falls nötig mit Intermediate-Zertifikaten installiert werden.

Der CryptoService nutzt mTLS mit gültigen Zertifikaten:

- Server-Zertifikat: CN/SAN auf localhost; Client-Zertifikate: CN clientX.
- Ablaufdaten sind gesetzt; Gültigkeit und Kette werden vom Truststore geprüft (Intermediate-CAs aktuell nicht erforderlich).
- Die Hostnamen-/Zertifikatsprüfung und Client-Auth laufen über die TLS-Schicht/Spring Security (u. a. X509AuthenticationFilter) gemäß SecurityConfig und application.properties.

Link zur SecurityConfig im GitHub-Repository (siehe Zeile 55-69):

<https://github.com/Marcey1611/Crypto-Microservice/blob/project-work/cryptoservice/src/main/java/com/projectwork/cryptoservice/boundary/security/SecurityConfig.java>

**Failed TLS connections should not fall back to an insecure connection [145]**

Wenn der TLS-Handshake fehlschlägt, darf keine unverschlüsselte Verbindung aufgebaut werden.

Der CryptoService erzwingt TLS über `server.ssl.enabled=true` in den `application.properties`. Bei fehlerhaftem Zertifikat, falscher mTLS-Authentifizierung oder einem anderen TLS-Problem wird die Verbindung sofort abgelehnt. Ein Fallback auf eine unverschlüsselte HTTP-Verbindung ist technisch nicht möglich. Die Konfiguration in SecurityConfig stellt sicher, dass alle Endpunkte ausschließlich über TLS erreichbar sind.

**Utilize TLS for all authenticated access and all other sensitive information [146]**

Sämtliche authentifizierten Zugriffe und alle Übertragungen sensibler Informationen müssen ausschließlich über TLS erfolgen.

Die komplette API des CryptoService ist per mTLS abgesichert. Alle sicherheitsrelevanten Endpunkte, `/generate-key`, `/generate-jwt`, `/encrypt` und `/decrypt`, sind ausschließlich über HTTPS erreichbar. Der TLS-Zwang wird in SecurityConfig.java konfiguriert, die Zertifikats- und Keystore-Einstellungen in application.properties hinterlegt.

Link zur application.properties im GitHub-Repository (Zeile 9-14):

<https://github.com/Marcey1611/Crypto-Microservice/blob/project-work/cryptoservice/src/main/resources/application.properties>

Link zur `SecurityConfig` im GitHub-Repository (siehe Zeile 55-69 genauer Zeile 58):  
<https://github.com/Marcey1611/Crypto-Microservice/blob/project-work/cryptoservice/src/main/java/com/projectwork/cryptoservice/boundary/security/SecurityConfig.java>

**Utilize a single standard TLS implementation that is configured appropriately [148]**

Es sollte nur eine zentrale TLS-Implementierung eingesetzt werden, die für die gesamte Anwendung einheitlich konfiguriert ist.

Der `CryptoService` nutzt eine einzige zentrale mTLS-Konfiguration, die in den `application.properties` hinterlegt ist. Diese TLS-Implementierung wird für alle Endpunkte gleichermaßen angewendet, es gibt keine Mischkonfigurationen oder unterschiedlichen TLS-Setups innerhalb des Services.

Link zur `application.properties` im GitHub-Repository (Zeile 9-14):  
<https://github.com/Marcey1611/Crypto-Microservice/blob/project-work/cryptoservice/src/main/resources/application.properties>

## 5. Bewertung und Reflexion

### 5.1. Reflexion

Im Rahmen der Projektarbeit konnten eine Vielzahl der OWASP SCPs erfolgreich umgesetzt werden. Dazu zählen insbesondere Maßnahmen aus den Bereichen Input-Validierung, Authentifizierung und Autorisierung, Kryptografie sowie Fehlerbehandlung und Logging. Die Umsetzung dieser SCPs führte zu einem durchgängig sicheren Kommunikationsfluss zwischen den beteiligten Clients und dem CryptoService.

Einige SCPs erwiesen sich als vergleichsweise einfach zu implementieren, etwa die konsequente Eingabevalidierung mit dedizierten Validator-Klassen oder das Logging von sicherheitsrelevanten Ereignissen unter Berücksichtigung des Datenschutzes.

Andere SCPs hingegen waren deutlich aufwendiger. Hierzu zählt beispielsweise die sichere Schlüsselverwaltung mit serverseitiger Speicherung, die JWT-basierte Autorisierung und die Absicherung aller Schnittstellen über mTLS. Diese Anforderungen erforderten eine saubere Architekturtrennung sowie ein durchdachtes Key-Management-Konzept, um Fehlkonfigurationen oder ungewollte Sicherheitslücken zu vermeiden.

Insgesamt hat sich gezeigt, dass die frühzeitige Berücksichtigung von Sicherheitsaspekten bei der Architekturplanung und der kontinuierliche Abgleich mit den relevanten SCPs entscheidend für den Umsetzungserfolg waren.

### 5.2. Grenzen

Nicht alle theoretisch relevanten SCPs konnten im Rahmen dieser Arbeit umgesetzt werden. Manche erwiesen sich als für den Anwendungsfall nicht sinnvoll oder nicht praktikabel, andere wären nur mit erheblichem Mehraufwand umsetzbar gewesen. Dazu zählen beispielsweise weitergehende Mechanismen zur Erkennung von Anomalien im Laufzeitverhalten, komplexe Audit- und Monitoring-Lösungen oder der Einsatz von Hardware-Sicherheitsmodulen (HSM) zur Schlüsselverwaltung.

Die Umsetzung der gewählten SCPs war zeit- und ressourcenintensiv, insbesondere bei sicherheitskritischen Funktionen wie dem Key-Management oder der Validierung und Signaturprüfung von JWTs. Für die vollständige Implementierung aller potenziell anwendbaren SCPs wäre ein noch höherer Planungs- und Entwicklungsaufwand erforderlich.

Gleichzeitig hat sich gezeigt, dass es langfristig realistisch und praktikabel ist, die wichtigsten SCPs von Beginn an bei jedem Commit einzuhalten, sofern sie fest in den Entwicklungsprozess integriert werden. Auf diese Weise ließe sich der Implementierungsaufwand verteilen, ohne die Projektlaufzeit unverhältnismäßig zu verlängern.

### 5.3. Ausblick und mögliche Erweiterungen

Für einen produktiven Einsatz des CryptoService wären mehrere funktionale und organisatorische Erweiterungen sinnvoll:

- Persistente Speicherung der Nutzerinformationen, um die Autorisierung nicht nur temporär, sondern dauerhaft verwalten zu können.
- Dockerisierung des gesamten Systems, um eine vereinfachte Bereitstellung, Skalierbarkeit und reproduzierbare Entwicklungsumgebungen zu ermöglichen.
- Optimierte Management von Keystore-Passwörtern, beispielsweise durch den Einsatz sicherer Secrets-Management-Lösungen.
- Ergänzung weiterer SCPs, wie erweiterte Protokollierung für sicherheitsrelevante Aktionen, automatisierte Sicherheitsprüfungen im Build-Prozess oder die Integration von Penetration-Testing-Tools in die Entwicklungs- und Testphase.

Darüber hinaus könnte das Projekt in einer zweiten Iteration um zusätzliche Sicherheitsmechanismen ergänzt werden, etwa durch rollenbasierte Zugriffskontrolle (RBAC) oder die Integration in ein zentrales Identity- und Access-Management-System.

Unabhängig vom konkreten Projekt könnten zukünftige Untersuchungen in diesem Bereich aufzeigen, wie sich Security-by-Design-Prinzipien noch enger in agile Entwicklungsprozesse integrieren lassen, welche Trade-offs zwischen Sicherheit und Performance in produktiven Systemen entstehen und wie sich automatisierte Codeanalyse zur kontinuierlichen Überprüfung der SCP-Konformität nutzen lässt.

# Literaturverzeichnis

Amazon Web Services, Inc., o.J.. *Microservices*. [Online]

Available at: [https://aws.amazon.com/microservices/?utm\\_source=chatgpt.com](https://aws.amazon.com/microservices/?utm_source=chatgpt.com)

[Zugriff am 01 08 2025].

Civardi, C., 2025. *7 Key Benefits of Microservices Architecture for Modern Applications*. [Online]

Available at: [https://blog.payara.fish/benefits-of-microservices-architecture?utm\\_source=chatgpt.com](https://blog.payara.fish/benefits-of-microservices-architecture?utm_source=chatgpt.com)

[Zugriff am 01 08 2025].

Cloudflare, Inc., o.J.. *What is mutual TLS (mTLS)?*. [Online]

Available at: [https://www.cloudflare.com/learning/access-management/what-is-mutual-tls/?utm\\_source=chatgpt.com](https://www.cloudflare.com/learning/access-management/what-is-mutual-tls/?utm_source=chatgpt.com)

[Zugriff am 09 08 2025].

EJBCA by Keyfactor, 2023. *Understanding mTLS and Its Role in Zero Trust Security*. [Online]

Available at: [https://www.ejbca.org/resources/understanding-mtls-and-its-role-in-zero-trust-security/?utm\\_source=chatgpt.com](https://www.ejbca.org/resources/understanding-mtls-and-its-role-in-zero-trust-security/?utm_source=chatgpt.com)

[Zugriff am 09 08 2025].

Gasiba, T. E., Lechner, U., Pinto-Albuquerque, M. & Mendez, D., 2021. *Is Secure Coding Education in the Industry Needed?*. [Online]

Available at: <https://arxiv.org/pdf/2102.05343>

[Zugriff am 01 08 2025].

Harris, C., o.J.. *What are microservices? Guide to modern application architecture*. [Online]

Available at: [https://www.atlassian.com/microservices/microservices-architecture?utm\\_source=chatgpt.com](https://www.atlassian.com/microservices/microservices-architecture?utm_source=chatgpt.com)

[Zugriff am 01 08 2025].

Ibrahim, M., 2024. *What is a JWT? Understanding JSON Web Tokens*. [Online]

Available at: [https://supertokens.com/blog/what-is-jwt?utm\\_source=chatgpt.com](https://supertokens.com/blog/what-is-jwt?utm_source=chatgpt.com)

[Zugriff am 01 08 2025].

IEEE, o.J.. *What is software security and why is it important?*. [Online]

Available at: <https://www.computer.org/resources/software-security>

[Zugriff am 01 08 2025].

Kampanakis, P. et al., 2024. *Practical Challenges with AES-GCM and*. [Online]

Available at: [https://csrc.nist.gov/csrc/media/Events/2023/third-workshop-on-block-cipher-modes-of-operation/documents/accepted-papers/Practical%20Challenges%20with%20AES-GCM.pdf?utm\\_source=chatgpt.com](https://csrc.nist.gov/csrc/media/Events/2023/third-workshop-on-block-cipher-modes-of-operation/documents/accepted-papers/Practical%20Challenges%20with%20AES-GCM.pdf?utm_source=chatgpt.com)

[Zugriff am 01 08 2025].

Lewis, J. & Fowler, M., 2014. *Microservices*. [Online]

Available at: [https://martinfowler.com/articles/microservices.html?utm\\_source=chatgpt.com](https://martinfowler.com/articles/microservices.html?utm_source=chatgpt.com)

[Zugriff am 01 08 2025].

Microsoft, 2025. *Microservices architecture style*. [Online]

Available at: [https://learn.microsoft.com/en-us/azure/architecture/guide/architecture-styles/microservices?utm\\_source=chatgpt.com](https://learn.microsoft.com/en-us/azure/architecture/guide/architecture-styles/microservices?utm_source=chatgpt.com)

[Zugriff am 01 08 2025].

OWASP, 2010. *OWASP Secure Coding Practices Quick Reference Guide*. [Online]  
Available at: [https://owasp.org/www-project-secure-coding-practices-quick-reference-guide/assets/docs/OWASP\\_SCP\\_Quick\\_Reference\\_Guide\\_v21.pdf](https://owasp.org/www-project-secure-coding-practices-quick-reference-guide/assets/docs/OWASP_SCP_Quick_Reference_Guide_v21.pdf)  
[Zugriff am 01 08 2025].

OWASP, o.J.. *Secure Coding Practices*. [Online]  
Available at: <https://owasp.org/www-project-secure-coding-practices-quick-reference-guide/stable-en/02-checklist/05-checklist>  
[Zugriff am 01 08 2025].

Peplinski, J., 2025. *Top 7 characteristics of microservices architecture*. [Online]  
Available at: [https://alokai.com/blog/microservices-characteristics?utm\\_source=chatgpt.com](https://alokai.com/blog/microservices-characteristics?utm_source=chatgpt.com)  
[Zugriff am 01 08 2025].

PyCryptodome, o.J.. *Modern modes of operation for symmetric block ciphers*. [Online]  
Available at:  
[https://pycryptodome.readthedocs.io/en/latest/src/cipher/modern.html?utm\\_source=chatgpt.com](https://pycryptodome.readthedocs.io/en/latest/src/cipher/modern.html?utm_source=chatgpt.com)  
[Zugriff am 01 08 2025].

Richardson, C., o.J.. *Microservice Architecture pattern*. [Online]  
Available at: [https://microservices.io/patterns/microservices.html?utm\\_source=chatgpt.com](https://microservices.io/patterns/microservices.html?utm_source=chatgpt.com)  
[Zugriff am 01 08 2025].

Sharifi, H. R., 2025. *Java AES Encryption and Decryption*. [Online]  
Available at: <https://www.baeldung.com/java-aes-encryption-decryption>  
[Zugriff am 01 08 2025].

Sumanth, G., 2024. *The Impact and Importance of Secure Coding Training*. [Online]  
Available at: <https://www.appsecengineer.com/blog/the-impact-and-importance-of-secure-coding-training>  
[Zugriff am 01 08 2025].

Timbó, R., 2025. *Security in Software Development*. [Online]  
Available at: <https://www.revelo.com/blog/security-in-software-development>  
[Zugriff am 01 08 2025].

Verhelst, L., 2025. *Mutual TLS (mTLS): A Deep Dive into Secure Client-Server Communication*. [Online]  
Available at: <https://medium.com/%40LukV/mutual-tls-mtls-a-deep-dive-into-secure-client-server-communication-bbb83f463292>  
[Zugriff am 09 08 2025].

Wikipedia, 2024. *Cloud-native Computing*. [Online]  
Available at: [https://de.wikipedia.org/wiki/Cloud-native\\_Computing?utm\\_source=chatgpt.com](https://de.wikipedia.org/wiki/Cloud-native_Computing?utm_source=chatgpt.com)  
[Zugriff am 01 08 2025].

Wikipedia, 2025. *Galois/Counter Mode*. [Online]  
Available at: [https://en.wikipedia.org/wiki/Galois/Counter\\_Mode?utm\\_source=chatgpt.com](https://en.wikipedia.org/wiki/Galois/Counter_Mode?utm_source=chatgpt.com)  
[Zugriff am 01 08 2025].

Wikipedia, 2025. *Microservices*. [Online]

Available at: [https://de.wikipedia.org/wiki/Microservices?utm\\_source=chatgpt.com](https://de.wikipedia.org/wiki/Microservices?utm_source=chatgpt.com)

[Zugriff am 01 08 2025].

# Anhang

## Anhang A: Dokumentation der Verwendung von KI-Werkzeugen

Die folgende Tabelle veranschaulicht die zur Erarbeitung dieser Arbeit verwendeten KI-Werkzeuge.

| <b>Zweck der Verwendung</b>                    | <b>Eingesetztes KI-Werkzeug</b>             | <b>Verwendungsweise bzw. -ort</b>                                                                                                                                                                                                                                                            |
|------------------------------------------------|---------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Unterstützung bei der sprachlichen Optimierung | ChatGPT (GPT-4o, GPT-5)                     | Vorschläge zur Verbesserung von Formulierungen, Klarheit und wissenschaftlichem Stil. Einzelne Absätze wurden auch vollständig zur Umformulierung vorgeschlagen, jedoch stets durch eigene Prüfung und Bearbeitung in den finalen Text übernommen. Einsatz verteilt über die gesamte Arbeit. |
| Strukturelles Feedback und Gliederungshinweise | ChatGPT (GPT-4o, GPT-5)                     | Unterstützung bei der Beurteilung und Optimierung der Gliederung und Kapitelstruktur. Entscheidungen wurden eigenständig abgewogen und umgesetzt.                                                                                                                                            |
| Rechercheunterstützung und Quellensuche        | Perplexity (Sonar), ChatGPT (GPT-4o, GPT-5) | Unterstützung bei der Identifikation relevanter Quellen und Publikationen. Alle verwendeten Quellen wurden eigenständig geprüft, eingeordnet und korrekt zitiert.                                                                                                                            |